

Guardian Connect

System Audit Report

Full Stack Code & Infrastructure Audit

February 2026

LIVE STATUS	
Backend API	ONLINE - back-end-production-4a69.up.railway.app
Web Frontend	ONLINE - web-user-production.up.railway.app
PostgreSQL	ONLINE - Connected
Database Health	OK - Responding
SERVICES	
Backend	Node.js/Express + TypeScript + Socket.io
Web User	React + Vite + TypeScript
Admin Panel	React + Vite + TypeScript + React Query
Database	PostgreSQL (Railway)
Invoice System	Next.js (can be removed)

Do We Need All These Services?

Short answer: Yes, but the invoice-system can go.

Service	Needed?	Why
Backend	YES	Core API, auth, emergency logic, Socket.io real-time, all business logic lives here
PostgreSQL	YES	All data storage - users, emergencies, contacts, messages, sessions
Web User	YES	The main user-facing app (emergency triggers, contacts, chat, location sharing)
Admin Panel	OPTIONAL	Useful for user management and messaging, but could be merged into web-user as an admin route
Invoice System	NO	Separate Next.js app, not part of core guardian-connect. Remove when ready.

The admin panel could eventually be merged into the web-user app as protected admin routes, saving one Railway service. But for now it works fine as a separate app.

Critical Issues (Fix Immediately)

1. BUG: .isEmpty on strings doesn't exist in JavaScript

Location: routes/emergencies.ts:88, services/push.ts:111

Code calls .isEmpty and .isNotEmpty on strings, which don't exist. FCM token validation is completely broken - emergency push notifications won't work. Fix: use .length > 0 instead.

2. Hardcoded JWT secret fallback

Location: services/jwt.ts:9

Falls back to 'default-secret-change-in-production' if env var missing. Anyone reading the source code can forge auth tokens. Fix: throw an error if JWT_SECRET is not set.

3. Overly permissive CORS

Location: server.ts:37-52

Regex /\.railway\.app\$/ allows ANY Railway subdomain to access your API. Combined with credentials: true, this enables CSRF attacks. Fix: use exact origin URLs only.

4. Hardcoded test credentials in production

Location: web-user Login.tsx:8-9, admin Login.tsx:7-8

Login forms pre-filled with user1@example.com / password123. Visible in production bundle. Fix: remove or wrap in if (import.meta.env.DEV).

5. JWT tokens stored in localStorage

Location: web-user Login.tsx:48-49, App.tsx:19

Access and refresh tokens in localStorage are vulnerable to XSS. Any injected script can steal them. Fix: use HttpOnly cookies or add CSP headers.

Security Issues

1. Socket.io CORS set to wildcard

services/socket.ts

Socket.io allows connections from ANY origin. Should match the HTTP CORS whitelist.

2. File uploads trust client MIME types

routes/emergencies.ts

No magic byte validation on uploaded files. A malicious file could be uploaded disguised as an image.

3. Rate limiting disabled in development

server.ts

Rate limiter only active in production. Easy to forget to re-enable.

4. No input validation on escalate endpoint

routes/emergencies.ts

Missing express-validator checks on emergency escalation.

5. JWT decoded without signature validation on frontend

web-user/utis/jwt.ts

Frontend decodes JWT payload directly. Never use decoded values for authorization decisions.

6. Redis initialized but never used

database/redis.ts

Redis client created on startup but not connected to any feature. Unnecessary attack surface.

Performance Issues

1. N+1 query in emergency creation

Backend loops through contacts and queries each FCM token individually. Should batch into a single query: `SELECT fcm_token FROM users WHERE id = ANY($1)`.

2. Frontend polls every 5 seconds even in background

Home.tsx polls `checkActiveEmergency()` and `checkPendingEmergencies()` on a 5s interval even when tab is hidden. Use Page Visibility API to pause.

3. 2,219-line dead code file in bundle

`EmergencyActive.OLD.tsx` is still in the source tree and gets bundled. Delete it immediately - it nearly doubles the page bundle size.

4. 3 unused npm packages in backend

`aws-sdk` (2MB+), `twilio`, and `redis` are in dependencies but never used. Remove them to cut install time and attack surface.

5. Missing database indexes

No composite index on `emergency_locations(emergency_id, user_id, timestamp)`. Location queries will slow down as data grows.

6. Short database connection timeout

2-second connection timeout in `db.ts` is too aggressive. Under load this will cause unnecessary failures. Increase to 5-10 seconds.

7. Hardcoded 100 message limit

Always fetches 100 messages per emergency even if fewer exist. Add pagination.

Code Quality & Architecture

Code Duplication (~30%)

The admin and web-user apps duplicate API setup, auth logic, login pages, and routing patterns. The API base URL is defined in 5+ different places across both apps. Recommendation: create a shared config.ts or eventually merge into a monorepo.

Inconsistent State Management

Web-user uses raw useState everywhere with no global state. Admin uses React Query (TanStack Query). Both should standardize on one approach - React Query is the better choice.

Inline Styles Throughout

Components use inline style objects instead of CSS modules or a styling library. Makes theming and maintenance difficult. Consider Tailwind CSS or CSS modules.

TypeScript Strictness: Low

Many 'any' types throughout the codebase. Strict mode not enabled. This defeats the purpose of TypeScript and allows bugs to slip through.

No Tests

Zero test files found across the entire project. No unit tests, integration tests, or E2E tests. For an emergency response app, this is a significant risk.

No Error Boundaries

Neither frontend has React Error Boundary components. A runtime error in any component crashes the entire app - dangerous for an emergency app.

No CI/CD Pipeline

No GitHub Actions or CI/CD configuration. Code goes directly from git push to production with no automated checks.

Excessive Console Logging

Backend logs 13+ lines per emergency creation, including sensitive data like token lengths and user emails. Frontend has debug logging throughout. Add environment-based log levels.

What's Working Well

- Backend builds clean with no TypeScript errors
- Health endpoint verifies database connectivity
- Socket.io real-time communication properly initialized
- Feature flags for donations/subscriptions (clean toggling)
- Retry logic with exponential backoff in web-user API service
- Protected routes prevent unauthorized access on both apps
- Service Worker registration for PWA support
- Admin uses React Query for efficient data fetching
- Graceful shutdown handling (SIGTERM/SIGINT)
- Database schema is well-designed with proper foreign keys and indexes
- Railway deployment configs are correct and services are running

Recommended Fix Priority

IMMEDIATE (This Week)

#	Task	Why
1	Fix .isEmpty bug	Push notifications completely broken
2	Remove hardcoded JWT fallback	Auth can be forged
3	Remove hardcoded test credentials	Visible in production
4	Delete EmergencyActive.OLD.tsx	2,219 lines of dead code
5	Remove unused packages (aws-sdk, twilio, redis)	Bloat + attack surface
6	Tighten CORS to exact origins	CSRF risk

SHORT TERM (Next 2 Weeks)

#	Task	Why
7	Fix N+1 queries in emergency creation	Performance under load
8	Add Error Boundaries to both frontends	App crashes gracefully
9	Centralize API URL config	Currently in 5+ places
10	Pause polling when tab is hidden	Battery/bandwidth waste
11	Add proper file upload validation	Security risk
12	Move tokens to HttpOnly cookies	XSS protection

MEDIUM TERM (Next Month)

#	Task	Why
13	Add test suite (Jest + React Testing Library)	Zero test coverage currently
14	Set up CI/CD with GitHub Actions	No automated checks
15	Enable TypeScript strict mode	Catch bugs at compile time
16	Add environment-based logging	Too much noise in prod
17	Consider merging admin into web-user	Reduce Railway costs
18	Remove invoice-system service	Not part of guardian-connect